



**Assessment Cover Sheet and Feedback Form
Examination 2025**

Course Code: UC2OOP101	Course Title: Object Oriented Programming	Lecturer: Johan van Niekerk
Assignment No: 3 of 3	Total number of pages, including this page: 5	Maximum Word Count: Not specified
Assignment Title: Object Oriented Programming – Report on Laboratory Work		
Date Released: 24.11.2025	Submission Date: 05.12.2025	Feedback Date: 26.12.2025

Section 1: Submission

Record of Submission and Plagiarism Declaration

In submitting this work you declare that this assignment is your own work and that any collaborative work or existing material has been specifically indicated in the text. In submitting this assignment you agree that this work may be submitted for plagiarism testing. Please note that this is an individual assignment and students are supposed to work on their own.

Work should be submitted:

- Electronically Via Moodle in the first case. If problems are encountered then via email to the course staff.

Assignments submitted in digital form MUST adhere to all requirements outlined in this document and only file formats as outlined in the 'what to submit' section will be accepted.

Late submissions will result in zero grade, unless agreed with course leader in advance.

IT IS YOUR RESPONSIBILITY TO KEEP A COPY OF ALL SUBMITTED WORK.

Section 2: Assignment Details

Your task is to demonstrate your grasp of Object-Oriented Programming (OOP) Principles and your skill in applying these principles using OOP Design Patterns. You'll achieve this by developing a proof-of-concept simulation that showcases these skills.

Scenario: – CityRide Micro-Mobility Fleet

CityRide operates a large network of shared **electric scooters and e-bikes** across multiple cities. To remain competitive, CityRide's management wants to create a **flexible simulation system** that models how rides, pricing, and vehicle availability might evolve — and can easily be extended when new city rules or subscription plans are introduced. You have been asked by the **Head of Software Engineering** to develop a **proof-of-concept prototype** that demonstrates how **Object-Oriented Design Principles** and **Design Patterns** can make such a system easy to **extend and maintain**.

System Overview

Your simulation should **demonstrate how an extensible design might support:**

Vehicles

- Each vehicle has a unique ID, battery level, and a location.
- Two initial types exist: Scooter and Bike.
- Both share common behavior (start ride, end ride, check status) but may differ in battery use and rules.

Riders

- Riders have a name and a **subscription plan** (e.g., "Pay-as-you-go", "Commuter", or "Tourist").
- Each plan may use different **pricing and penalty strategies** (e.g., surge pricing, off-peak discounts, overstay fees).

Rides

- Rides begin and end in a **Zone** (e.g., Downtown, Campus, Suburb).
- Zones may have different pricing rules or restrictions (e.g., "no-ride zones").
- When a ride ends, the system should **trigger events** (e.g., billing, data logging).

Billing

- Bills are generated automatically when a ride ends.
- Pricing depends on the plan and the zone.
- Riders can optionally add **add-ons** (insurance, helmet rental, etc.) that affect cost.

Extensibility

CityRide plans to expand its services with:

- **New vehicle types** (cargo bike, e-moped)
- **New plans** (corporate, student)
- **New pricing behaviors** (flat-rate, time-of-day, dynamic).

Your design should make these additions possible **without rewriting core classes**.

Your Task

Create a **simulation prototype** and an accompanying **written report** that demonstrate how well-designed OO architecture supports flexibility and future-proofing. Your **target audience** is the **CityRide software development team**, who have **limited OO experience**.

The simulation should be similar in scope to the **SimUDuck** example used in class — i.e., focused on **dynamic algorithm changes, object communication, and coupling management**, rather than complex data handling.

You may either:

- Develop **one integrated simulation** addressing all design questions, or
- Develop **separate small simulations**, each illustrating one question.

Design Questions

Your report must explicitly address the **four guiding design questions** used in this course. Each answer must appear under its own heading and include:

1. Which **OO principles** and/or **design patterns** are applied.
2. How these choices reduce **coupling**, improve **extensibility**, or support **maintainability**.
3. How these choices make the system more **resistant to future change**.
4. Example code snippets from your simulation that demonstrate the concept.

You are encouraged to propose **creative extensions** (e.g., loyalty points, eco-bonus programs, or maintenance bots) that help demonstrate your understanding of OO principles and patterns.

The assignment is **intentionally open-ended** to allow innovative, well-justified design choices.

Restrictions

1. You may **not use any additional Python libraries** other than abc (or simple utility libraries like numpy or collections)
2. The simulation must be **text-based only** (no GUI or external data).
3. The notebook must **run sequentially from top to bottom without errors**.
4. The focus is on demonstrating OO design concepts, not implementing a real operational system.
5. You **may not use AI tools** to complete this assignment

What to Submit

You must submit a **single Jupyter notebook** (.ipynb) that contains both your simulation prototype and written report.

Code requirements

- Include all **class definitions** and **interfaces** relevant to your design.
- Include **simulation examples** that demonstrate:
 - How your system supports new vehicles/plans with minimal change.
 - How pattern-based designs make code modular and maintainable.
- Ensure all code cells execute correctly and logically in sequence.

Written report requirements

- Embedded in the same notebook, between code cells (in Markdown).
- Should clearly explain:
 - **The design choices** made and how they relate to course-taught principles/patterns.
 - **Why these choices make the code future-proof** and resistant to change.
 - **How to use your prototype** (e.g., “To view how the Strategy pattern enables dynamic pricing, run the following cell...”).
 - **Reflection on assumptions** and potential extensions.
- Each design question must have its own heading and be linked to the code demonstrating it.

Notebook Layout Guidelines

To ensure clarity and consistency:

- Introduce each simulation example with a short markdown explanation indicating **which design question** it demonstrates.
- Follow each code cell that produces output with a **markdown cell explaining what the output shows**.
- Keep the notebook **fully executable top-down** (no broken cells).
- Upload any **images or diagrams** (class diagrams, UMLs, etc.) alongside the notebook as **separate, uncompressed files**.
- Include a **“Using Your Prototype”** section near the start or end of the notebook describing how to run key examples.
- Encourage **clear commenting and naming** for readability.
- You may demonstrate *extension* by adding a new class or behavior and showing that the rest of the system does not need to change.

Section 3: Marking and Assessment	
This assignment will be marked out of:	<u>100%</u>
Contribution to the final course mark:	<u>70%</u>
Estimated number of hours to be spent on this assignment:	<u>32</u>

The pass grade for this assignment is an E

Feedback for this assignment and a breakdown of the final grade will be provided via FS. The final course grade composed of this and any other assignment(s) will be awarded in the range A-F and recorded in the primary student information system (e.g. ECS).

Marking Criteria Guide	
Criteria	Factors that will be considered
Main logic and code professionalism (20 marks)	- Overall program structure and layout - Code is commented with explanations - Appropriate naming of classes, methods, etc. - Appropriate use of foundational OO concepts - Fully functional simulation program
Design Patterns and Principles (30 marks)	- All dependencies identified and addressed via appropriate application of design patterns and principles. - Many design patterns and principles used in well integrated overall design. - Design is mostly open for future extension whilst core elements are mostly encapsulated and closed for modification. - Patterns well integrated with the rest of the code without adding additional dependencies
Simulation Test Data and Scenarios (20 marks)	- How well do the selected scenarios demonstrate the extensibility of the code? - How easy to use is the simulation? (Could I just run each jupyter cell in sequence (or select menu items sequentially) and easily see exactly how and why the design is sensible?) - Is the case for each pattern demonstrated clearly?
Written Report and Explanation (30 marks)	- Assumptions and reflection on design fully documented. - All possible problems where future changes could negatively influence design has been identified and explained as if to a programmer who lacks OO experience and insight. All choices clearly explained and demonstrates the role(s) of chosen patterns and principles in ensuring ease of extensibility. All choices clearly explained and justified based on principles taught. - Reflection on design and justification of choices exceptionally well presented in a professionally formatted report. - Few / minor spelling mistakes. Clear well punctuated sentences. Writing style clear and informative without being verbose.

Mark Symbols:

F (<40)	E (40 – 49)	D (50 -59)	C (60-69)	C (70-79)	B (80-89)	A (90-100)
Fail	Sufficient	Satisfactory	Satisfactory-Good	Good	Very Good	Excellent